Spring Data JPA项目，当把一个方法的执行标记成事务执行，同时设置成readOnly的模式，会有什么风险或者潜在的问题 @Transactional(readOnly = true)
public void testTransaction() {
// jdbcTemplate.execute("select ...");
System.out.println("Done");
}

在Spring Data JPA中将方法标记为 `@Transactional(readOnly = true)` 主要存在以下风险和潜在问题：

1. 数据不一致风险

```java
@Transactional(readOnly = true)
public void testTransaction() {
    // ❌ 这里如果执行更新操作会引发异常
    jdbcTemplate.execute("UPDATE user SET name='test' WHERE id=1");
    // 抛出异常：Connection is read-only
}
```

- 尝试修改数据会抛出 `TransientDataAccessResourceException`
- 但某些数据库驱动可能忽略该设置，导致意外的数据修改

2. Hibernate 一级缓存问题

```java
@Transactional(readOnly = true)
public User findUser(Long id) {
    User user = userRepository.findById(id).orElse(null);
    // 尝试修改不会自动持久化到数据库
    user.setName("New Name");  // 修改只在缓存中，不会刷新到DB
    return user; // 返回的对象状态与数据库不一致
}
```

3. 懒加载异常风险

```java
@Transactional(readOnly = true)
public User getUserWithOrders(Long userId) {
    User user = userRepository.findById(userId).orElse(null);
    // 方法结束时Session关闭，但orders还未加载
    return user; // 事务结束后访问user.getOrders()会抛LazyInitializationException
}
```

4. FlushMode 相关陷阱

```java
@Transactional(readOnly = true)
public List<User> searchUsers(String name) {
    // 默认FlushMode为MANUAL，某些查询可能看不到未提交的修改
    List<User> users = userRepository.findByName(name);
    return users;
}
```

5. 性能误导

```java
@Transactional(readOnly = true)
public void testTransaction() {
    // 方法体内没有任何数据库操作
    System.out.println("Done");
    // 但仍然会开启事务，产生事务开销
}
```

- 空事务或不必要的事务会造成资源浪费
- 开启事务需要获取数据库连接，增加连接池压力

最佳实践建议

```java
```

```java
@Service
public class UserService {

    // ✅ 有实际查询操作时才使用
    @Transactional(readOnly = true)
    public User findUser(Long id) {
        return userRepository.findById(id).orElse(null);
    }

    // ✅ 明确需要修改的操作
    @Transactional
    public void updateUser(Long id, String name) {
        User user = userRepository.findById(id).orElse(null);
        user.setName(name);
        // 事务提交时会flush到数据库
    }

    // ✅ 无数据库操作的方法不加事务
    public void logOperation(String message) {
        System.out.println("Log: " + message);
    }
}
```

总结： `readOnly=true` 主要用于优化查询性能并防止意外修改，但应只在确实只读的操作上使用，避免不必要的开销和潜在问题。